

Explicação Completa da Aplicação MotoRent

Visão Geral do Projeto

MotoRent é um sistema completo de gerenciamento de locação de motos e vendas planejadas, desenvolvido com Flask (Python) no backend e HTML/CSS/JavaScript no frontend. O sistema inclui funcionalidades como dashboard com métricas, gerenciamento de clientes, veículos, pagamentos, multas, manutenção, telemetria, upload de documentos e verificação de background. Suporta autenticação de usuários com roles (admin/operador), modo claro/escuro, PWA (Progressive Web App) e OCR para processamento de documentos.

Estrutura de Arquivos

- `app.py`: Arquivo principal da aplicação Flask com todas as rotas, funções de banco de dados e lógica de negócio.
- `db_init.py`: Script de inicialização do banco de dados SQLite com schema e dados de exemplo.
- `requirements.txt`: Lista de dependências Python (Flask, Flask-Cors, Pillow, pytesseract, fpdf).
- `templates/`: Diretório com templates HTML (base.html, index.html, etc.).
- `static/`: Diretório com arquivos estáticos (CSS, JS, ícones, manifest.json, service-worker.js).
- `database.db`: Banco de dados SQLite.
- `uploads/`: Diretório para documentos enviados pelos usuários.
- `README.md`: Documentação do projeto.

Backend (app.py)

O arquivo `app.py` contém aproximadamente 900 linhas de código e implementa:

Configuração e Inicialização

- Configuração do Flask com CORS, sessão, upload de arquivos.
- Conexão com banco de dados SQLite.
- Configuração de PWA com manifest e service worker.

Autenticação

- Rotas para login, registro, logout, reset de senha.
- Sistema de roles (admin/operador) com permissões diferenciadas.
- Proteção de rotas com decoradores.

Gerenciamento de Clientes

- CRUD completo para clientes (criar, ler, atualizar, deletar).
- Sistema de pontuação de risco (score) para avaliação de crédito.
- Upload e processamento de documentos com OCR.

Gerenciamento de Veículos

- CRUD para veículos com informações como marca, modelo, placa, status, seguro, localização GPS.
- Integração com telemetria para rastreamento em tempo real.

Sistema Financeiro

- Gerenciamento de pagamentos, multas e vendas.
- Relatórios de receita, inadimplência e previsões.
- Integração com PDV (ponto de venda).

APIs

- Endpoints REST para dashboard, mapas, telemetria, etc.
- Suporte a AJAX para atualizações dinâmicas.

Outras Funcionalidades

- Sistema de manutenção preventiva.
- Verificação de background de clientes.
- Modo offline com service worker.
- Tema claro/escuro.

Banco de Dados (database.db)

O banco de dados SQLite contém as seguintes tabelas:

users

- id, username, email, password_hash, role
- Armazena usuários do sistema com autenticação.

customers

- id, name, document_type, document, email, phone, score, internal_notes
- Dados dos clientes com pontuação de risco.

vehicles

- id, brand, model, plate, color, status, insurance, latitude, longitude, owner_id
- Informações dos veículos com localização GPS.

payments

- id, customer_id, vehicle_id, amount, due_date, status, payment_method
- Registros de pagamentos e aluguéis.

fines

- id, customer_id, vehicle_id, amount, description, date, status, points
- Multas aplicadas aos clientes.

maintenance

- id, vehicle_id, last_service_date, next_service_date, status, predicted_cost, note
- Agendamento de manutenções.

telemetry

- id, vehicle_id, latitude, longitude, speed, timestamp, status
- Dados de telemetria para rastreamento.

documents

- id, owner_type, owner_id, doc_type, filename, note, background_check, uploaded_at
- Documentos enviados (CNH, CRLV, etc.).

sales

- id, customer_id, vehicle_id, sale_price, installments, installment_value, sale_date, status, payment_method
- Vendas de veículos.

Frontend (Templates e Static)

Templates HTML

- `base.html`: Template base com sidebar de navegação, topbar e layout responsivo.
- `index.html`: Dashboard com cards de métricas, gráficos e mapa.
- Outros templates: login, registro, gerenciamento de clientes/veículos, etc.

CSS (style.css)

- Design responsivo com suporte a modo claro/escuro.
- Variáveis CSS para temas.
- Layout em grid para sidebar e conteúdo principal.
- Animações e transições suaves.

JavaScript

- `app.js`: Lógica para dashboard, gráficos (Chart.js), mapa (Leaflet), atualizações AJAX.
- `forms.js`: Validação e manipulação de formulários.

Dependências (requirements.txt)

- Flask>=3.0: Framework web principal.
- Flask-Cors>=3.0: Suporte a CORS para APIs.

- Pillow>=10.0: Processamento de imagens para OCR.
- pytesseract>=0.3.10: OCR para extração de texto de documentos.
- fpdf>=1.7.2: Geração de PDFs.

Funcionalidades Principais

Dashboard

- Métricas em tempo real: total de clientes, veículos, aluguéis ativos, inadimplência, receita.
- Gráficos de receita mensal e lucro por veículo.
- Tabelas de alocação da frota e clientes de risco.
- Mapa interativo com localização dos veículos.

Gerenciamento

- Clientes: Cadastro com score de risco, upload de documentos.
- Veículos: Inventário com status, seguro, localização.
- Financeiro: Pagamentos, multas, vendas planejadas.
- Manutenção: Agendamento preventivo.
- Telemetria: Rastreamento GPS em tempo real.

Segurança e Usabilidade

- Autenticação robusta com hash de senhas.
- Controle de acesso baseado em roles.
- Interface responsiva e acessível.
- Suporte a PWA para uso offline.
- Tema adaptável (claro/escuro).

Como Executar

1. Instalar dependências: ``pip install -r requirements.txt``
2. Inicializar banco: ``python db_init.py``

3. Executar aplicação: ``python app.py``

4. Acessar em `http://localhost:5000`

Conclusão

MotoRent é uma aplicação completa para gestão de locadora de motos, combinando tecnologias modernas como Flask, SQLite, JavaScript, Chart.js e Leaflet para fornecer uma experiência rica e funcional tanto para administradores quanto para operadores.</content>

<parameter name="filePath">c:\Users\wasll\Desktop\Nova pasta\explicacao_completa.txt